Lab # 03

COURSE CODE: SE-206L

COURSE TITLE:

Software Construction & Development

LAB TITLE

JUnit Unit Testing

LAB ASSESSMENT TABLE:

Obtained Marks	Total Marks

Submitted To: Engr. Rizwan Shah.

Submitted By: Muhammad Danial.

Registration – No: 24ABSWE0021.

Semester: 5th

Date: 06-09-2026.

Dept. of Software Engineering

UET Peshawar, Abbottabad Campus.

Instructor Sign : _____

Lab 3:

JUnit Unit Testing

Repository Link:
https://github.com/danial-uet/Software-Construction-Labs./tree/master/Lab3

Homework 1 – Enhanced Calculator

Objective: Extend the in-lab Calculator1 class by adding advanced operations and ensuring comprehensive test coverage.

Tasks:

1. Add new methods:
   - power(int base, int exp)
   - modulus(int a, int b) → throws IllegalArgumentException if b = 0

2. Write JUnit 5 tests covering normal, boundary, and exceptional cases.
3. Use @BeforeEach and @AfterEach to initialize and reset the calculator instance.
4. Generate a code-coverage report in NetBeans and record the percentage.

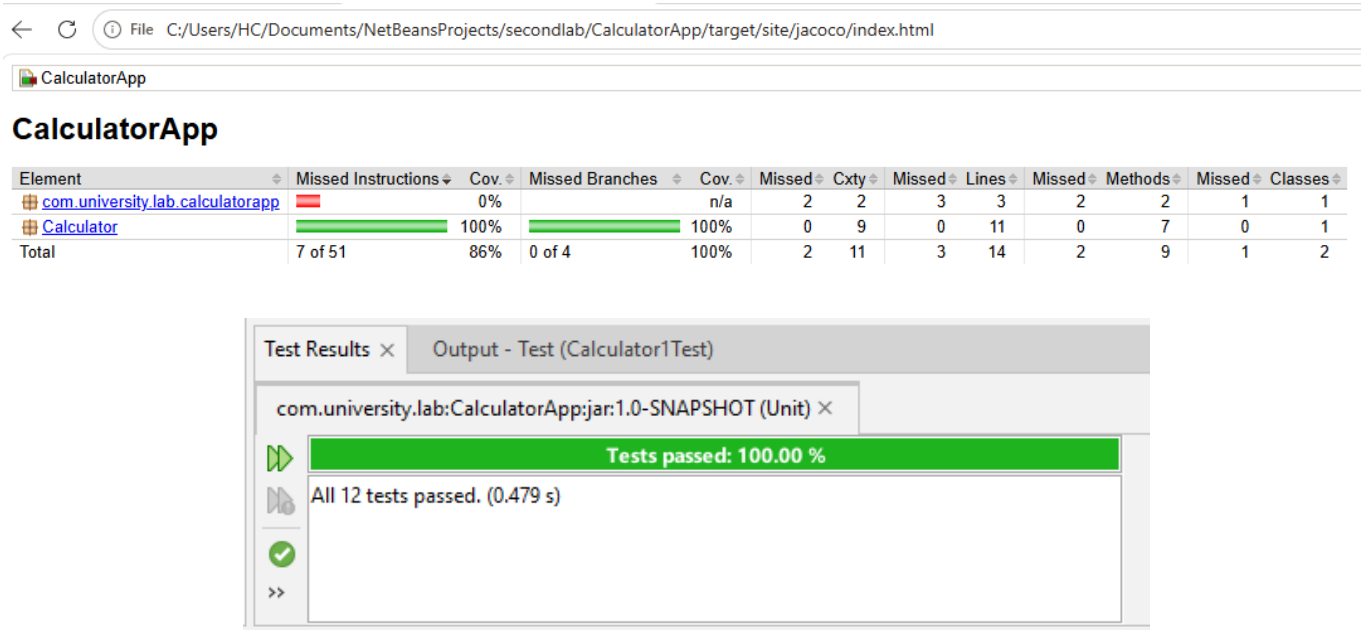
Solution

For first homework task, two separate, dedicated files were created by me inside Calculator Package in CalculatorApp Project:

● Calculator1.java, (Java Class):
https://github.com/danial-uet/Software-Construction-Labs./tree/master/Lab3/src/main/java/Calculator

● Calculator1Test.java, (Junit test class):
https://github.com/danial-uet/Software-Construction-Labs./tree/master/Lab3/src/test/java/Calculator

- Calculator1.java, Calculator1Test.java are in the GitHub repository.
- Coverage screenshot



- **Short reflection (≈ 100 words on what improved testing quality):**

  This homework improved my understanding of how systematic testing can increase the reliability of a program. I extended the calculator by adding power and modulus operations and tested normal, boundary, and exceptional cases. Using @BeforeEach and @AfterEach helped me manage the calculator object consistently for each test. The tests also helped verify that invalid operations, such as division or modulus by zero, were handled correctly. Running the complete test suite gave me greater confidence that the calculator behaves as expected. Overall, this exercise showed me that writing different types of test cases can identify errors more effectively than checking only normal inputs.

Homework 2 – Account Validation

Tasks

```
public void deposit(double amt);
public void withdraw(double amt);
public double getBalance();
public boolean isActive();
```

● Disallow negative deposits or withdrawals beyond balance.
● Mark inactive if balance < 100 after withdrawal.

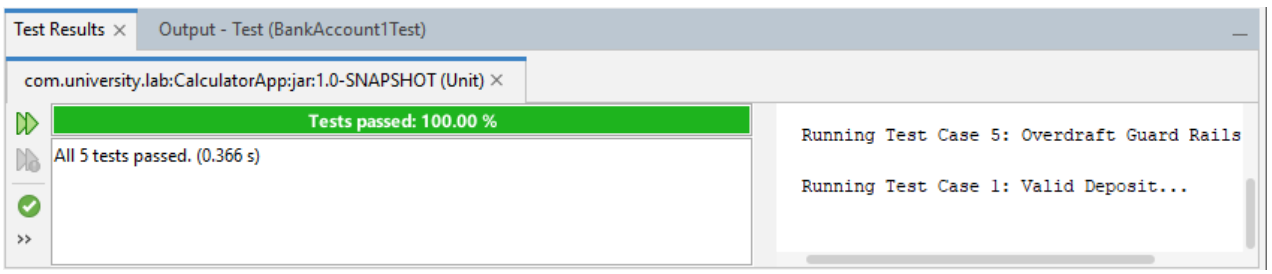
2. Write JUnit tests using assertThrows, assertTrue, assertFalse.
3. Add at least five distinct test cases: valid deposit, overdraft, inactivity, etc.
4. Submit test-case table + source files

For second task, two separate, dedicated files were created by me inside BankAccout Package in CalculatorApp Project:

● BankAccount1.java, (Java Class).
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab3/src/main/java/BankAccount/BankAccount1.java

● BankAccount1Test.java, (Junit test class).
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab3/src/test/java/BankAccount/BankAccount1Test.java

Output:



Test-case table

Test Case ID	Scenario Target	Input / Action	Expected Result	Assertion Method Used	Status
TC-01	Valid Deposit	Deposit $50.0 into $100.0 balance	Balance becomes $150.0	assertEquals()	**PASSED**
TC-02	Valid Withdrawal	Withdraw $50.0 from $200.0 balance	Balance becomes $150.0	assertEquals()	**PASSED**
TC-03	Inactivity Trigger	Withdraw $30.0 from $120.0 balance	Balance becomes $90.0; Status changes to Inactive	assertFalse()	**PASSED**
TC-04	Invalid Negative Inputs	Attempt negative actions (-$25.0, -$10.0)	Program throws IllegalArgumentException	assertThrows()	**PASSED**
TC-05	Overdraft Prevention	Withdraw $60.0 from $50.0 balance	Program throws IllegalArgumentException	assertThrows()	**PASSED**

Homework 3 – String Utility Testing

Objective:

Apply JUnit to verify correctness of string-processing logic.

Tasks

1. Implement StringAnalyzer with methods:

```
public boolean isPalindrome(String s);
public int countVowels(String s);
public boolean isAnagram(String s1, String s2);
```

2. Write JUnit tests that include:

- Empty or null inputs (should return false).
- Case-insensitive checks for palindromes/anagrams.
- Various vowel-count scenarios.

3. Use @DisplayName and meaningful test names.
4. Document results in a Test Case Summary Table and add one observation about potential edge-case handling.

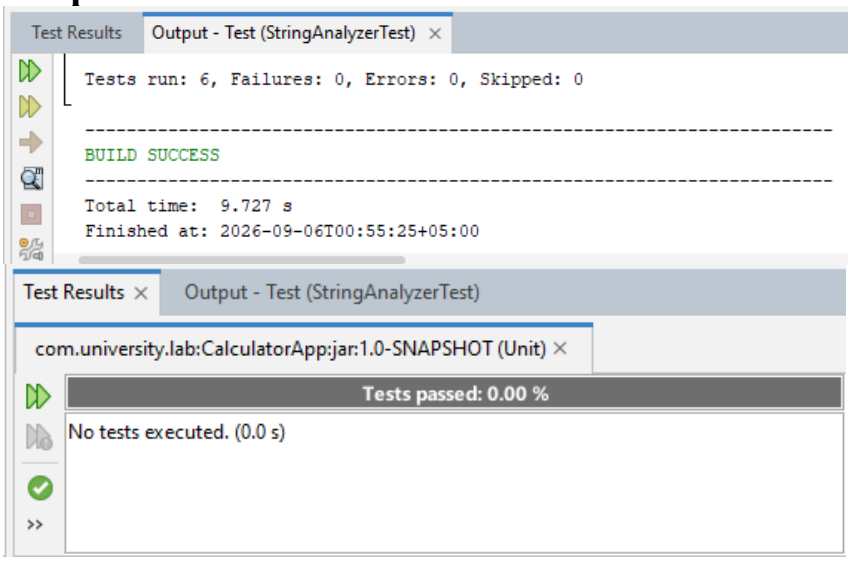
Solution

For third task, two separate, dedicated files were created by me inside BankAccout Package in CalculatorApp Project:

● StringAnalyzer.java, (Java Class).
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab3/src/main/java/BankAccount/StringAnalyzer.java

● StringAnalyzerTest.java, (Junit test class).
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab3/src/test/java/BankAccount/StringAnalyzerTest.java

Output



Test-case table

Test Reference ID	Target Objective	Input Parameters / Scenario	Expected Result	Assertion Method	Status
HW3-TC01	Palindrome Null Guard	null and empty string "" inputs	false	assertFalse()	PASSED
HW3-TC02	Palindrome Case Check	Mixed-case spaced phrase "RaceCar"	true	assertTrue()	PASSED
HW3-TC03	Vowel Count Variations	Mixed-case "AeIoU" & vowel-less "Rhythm"	5 and 0	assertEquals()	PASSED
HW3-TC04	Anagram Boundary Guard	Empty strings "" inside parameter fields	false	assertFalse()	PASSED
HW3-TC05	Anagram Case Check	Mixed-case inputs "Listen" and "Silent"	true	assertTrue()	PASSED

Required Edge-Case Observation (Task 4)

Our implementation successfully manages null and empty string ("") boundary inputs by checking for them at the very beginning of each method and returning false or 0 safely. This preventative strategy ensures the application remains stable under all conditions and completely prevents unexpected NullPointerException crashes during execution.

Post Lab Tasks:

Lab Task 1 — Temperature Converter

Objective:

Test correctness of temperature conversion formulas between Celsius, Fahrenheit, and Kelvin.

Implementation (TemperatureConverter.java)

```
public class TemperatureConverter {
    public double celsiusToFahrenheit(double c) { return (c * 9/5) + 32; }
    public double fahrenheitToCelsius(double f) { return (f - 32) * 5/9; }
    public double celsiusToKelvin(double c) { return c + 273.15; }
}
```

Test ideas (TemperatureConverterTest.java)

- Verify known pairs: 0 °C → 32 °F, 100 °C → 212 °F.
- Round-trip check: fahrenheitToCelsius(celsiusToFahrenheit(x)) ≈ x.
- Assert approximate equality using assertEquals(expected, actual, 0.01)

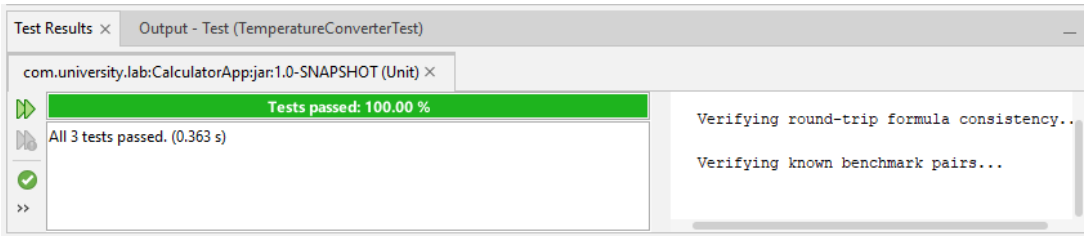
Solution

For first task, two separate, dedicated files were created by me inside ThirdLabTasks Package in CalculatorApp Project:

- TemperatureConverter.java, (Java Class).
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab3/src/main/java/ThirdLabTasks/TemperatureConverter.java

- TemperatureConverterTest.java, (Junit test class).
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab3/src/test/java/ThirdLabTasks/TemperatureConverterTest.java

Output



Test-case table

Test Case ID	Target Method / Rule	Input Action	Expected Output	Assertion Strategy	Status
TT-01	celsiusToFahrenheit	Input 0.0 °C & 100.0 °C	32.0 °F & 212.0 °F	assertEquals(exp, act, 0.01)	**PASSED**
TT-02	Round-Trip Accuracy	Loop 25.0 °C through both formulas	Approximately 25.0 °C	assertEquals(exp, act, 0.01)	**PASSED**
TT-03	celsiusToKelvin	Input 0.0 °C base value	273.15 K	assertEquals(exp, act, 0.01)	**PASSED**

Lab Task 2 — Bank Account Operations

Objective:

Test class invariants and exceptional conditions (negative balance, overdraft).

Implementation (BankAccount.java)

```
public class BankAccount
   { private double balance;
     public void deposit(double amt){ if(amt<0) throw new IllegalArgumentException();
balance+=amt; }
                public void withdraw(double amt){ if(amt>balance) throw
                            new IllegalStateException(); balance-=amt; }
   public double getBalance(){ return balance; }
}
```

Test ideas

- Deposit positive → balance increases.
- Withdraw valid → balance decreases.
- Withdraw beyond balance → expect IllegalStateException.
- Deposit negative → expect IllegalArgumentException.

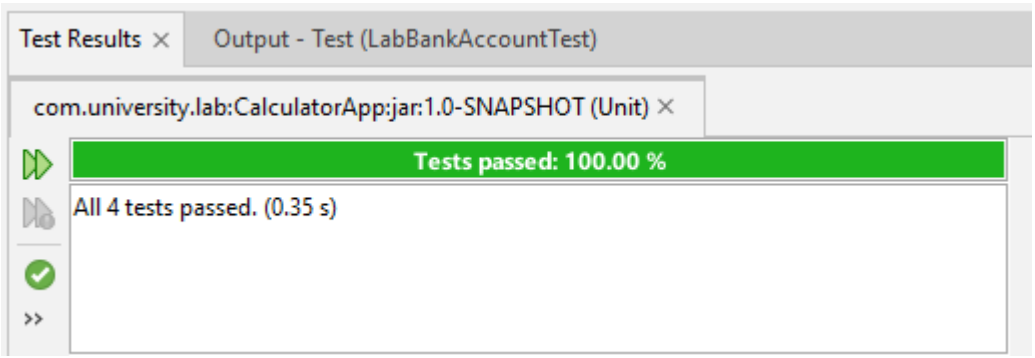
Solution

For second task, two separate, dedicated files were created by me inside ThirdLabTasks Package in CalculatorApp Project:

● LabBankAccount.java, (Java Class).
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab3/src/main/java/ThirdLabTasks/LabBankAccount.java

● LabBankAccountTest.java, (Junit test class).
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab3/src/test/java/ThirdLabTasks/LabBankAccountTest.java

Output



Test-case table

Test Reference ID	Target Objective	Input / Action	Expected Result	Assertion Method	Status
LT2-TC01	Deposit Verification	Deposit $50.0 into $100.0 account	Balance becomes $150.0	assertEquals()	**PASSED**
LT2-TC02	Withdrawal Verification	Withdraw $50.0 from $200.0 account	Balance becomes $150.0	assertEquals()	**PASSED**
LT2-TC03	Overdraft Condition	Withdraw $60.0 from $50.0 account	Throws IllegalStateException	assertThrows()	**PASSED**
LT2-TC04	Negative Input Guard	Deposit -$20.0 into account	Throws IllegalArgumentException	assertThrows()	**PASSED**

Lab Task 3 — String Utility (Palindrome Checker)

Objective:

Use JUnit to test logic and case sensitivity in string operations. Implementation (StringUtil.java)

```
public class StringUtil {
   public boolean isPalindrome(String s)
      { if(s==null) return false;
      String clean = s.replaceAll("[^A-Za-z]", "").toLowerCase();
      return new StringBuilder(clean).reverse().toString().equals(clean);
   }
}
```

Test ideas

- "madam" → true
- "RaceCar" (case insensitive) → true
- "hello" → false
- null or empty string → false

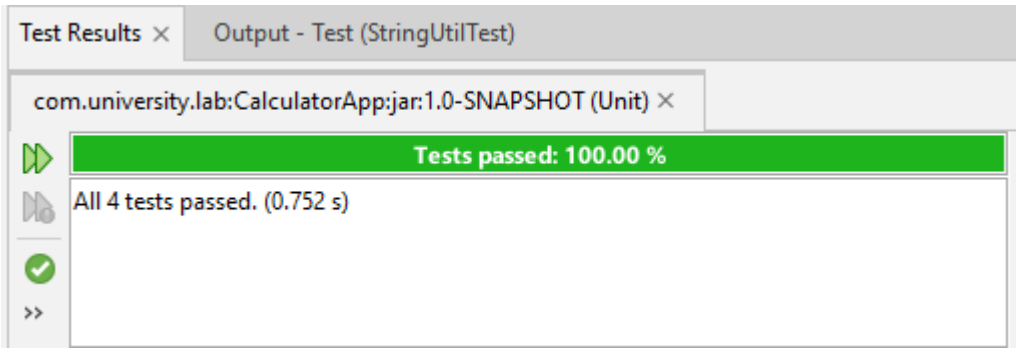
Solution

For third task, two separate, dedicated files were created by me inside ThirdLabTasks Package in CalculatorApp Project:

- StringUtil.java, (Java Class).
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab3/src/main/java/ThirdLabTasks/StringUtil.java

- StringUtilTest.java, (Junit test class).
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab3/src/test/java/ThirdLabTasks/StringUtilTest.java

Output



Test-case table

Test Reference ID	Target Objective	Input / Action	Expected Result	Assertion Method	Status
LT3-TC01	Standard Palindrome	Check string "madam"	true	assertTrue()	**PASSED**
LT3-TC02	Case Insensitive Match	Check mixed-case string "RaceCar"	true	assertTrue()	**PASSED**
LT3-TC03	Non-Palindrome Guard	Check normal string "hello"	false	assertFalse()	**PASSED**
LT3-TC04	Invalid Null / Empty Check	Check null and empty string ""	false	assertFalse()	**PASSED**

Lab Task 4 — Simple Timer Utility

Objective:
Test logic that handles elapsed-time computation and boundary conditions.

Implementation (TimerUtil.java)

```
public class TimerUtil {
   public int secondsBetween(int start, int end) {
      if(end < start) throw new IllegalArgumentException("End < start");
      return end - start;
   }
}
```

Test ideas

- Normal case: start = 10, end = 25 → 15 s.
- Boundary: start = 0, end = 0 → 0 s.
- Invalid input: expect IllegalArgumentException.

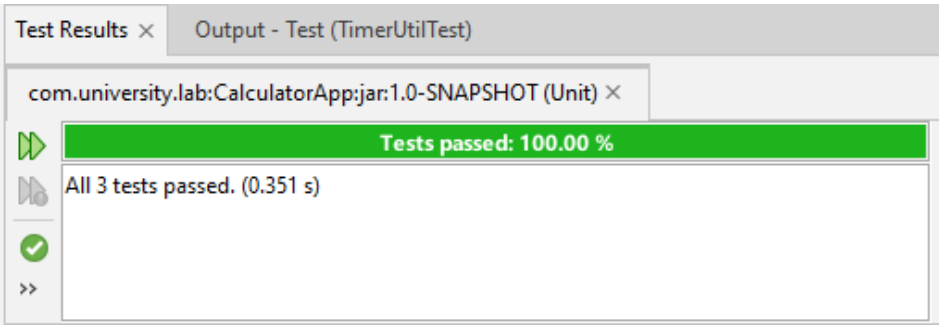
Solution

For forth task, two separate, dedicated files were created by me inside ThirdLabTasks Package in CalculatorApp Project:

- TimerUtil.java, (Java Class).
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab3/src/main/java/ThirdLabTasks/TimerUtil.java

- TimerUtilTest.java, (Junit test class).
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab3/src/test/java/ThirdLabTasks/TimerUtilTest.java

Output



Test-case table

Test Reference ID	Target Objective	Input Parameters	Expected Result	Assertion Method	Status
LT4-TC01	Normal Duration	start = 10, end = 25	15	assertEquals()	**PASSED**
LT4-TC02	Zero Boundary Offset	start = 0, end = 0	0	assertEquals()	**PASSED**
LT4-TC03	Time Conflict Guard	start = 30, end = 15	Throws IllegalArgumentException	assertThrows()	**PASSED**

Lab Task 5 — Shopping Cart (Mini Case Study)

Objective:

Combine multiple test assertions and simulate business rules.

Implementation (ShoppingCart.java)

```
import java.util.*;
public class ShoppingCart {
   private final List<String> items = new ArrayList<>();
   public void addItem(String item){ items.add(item); }
   public void removeItem(String item){ items.remove(item); }
   public int getItemCount(){ return items.size(); }
   public void clear(){ items.clear(); }
}
```

Test ideas

- Add 3 items → count = 3.
- Clear cart → count = 0.
- Remove 1 item → count = 2.
- Removing a non-existent item doesn't throw an error.

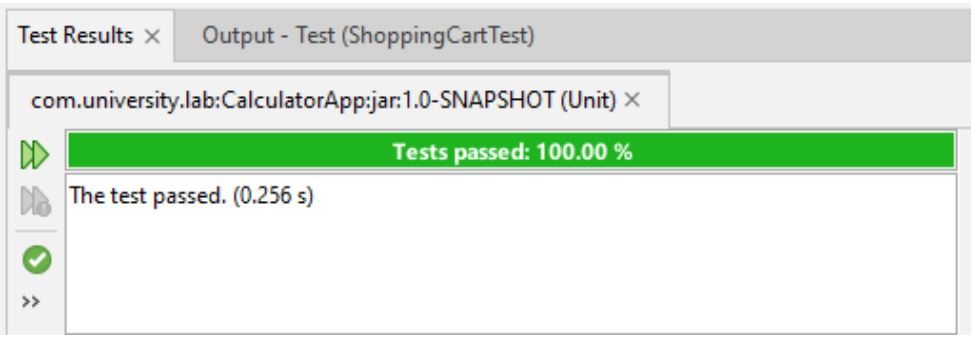
Solution

For fifth task, two separate, dedicated files were created by me inside ThirdLabTasks Package in CalculatorApp Project:

● ShoppingCart.java, (Java Class).
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab3/src/main/java/ThirdLabTasks/ShoppingCart.java

● ShoppingCartTest.java, (Junit test class).
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab3/src/test/java/ThirdLabTasks/ShoppingCartTest.java

Output



Test-case table

Test Reference ID	Target Objective	Action Sequence	Expected Result	Assertion Method	Status
LT5-TC01	Item Addition	Add 3 distinct items to cart	getItemCount() returns 3	assertEquals()	**PASSED**
LT5-TC02	Item Removal	Remove 1 existing item from cart	getItemCount() drops to 2	assertEquals()	**PASSED**
LT5-TC03	Missing Item Handling	Attempt removing an item not in cart	No exceptions thrown; count stays 2	assertDoesNotThrow()	**PASSED**
LT5-TC04	Cart Reset	Invoke clear operation on active cart	getItemCount() returns 0	assertEquals()	**PASSED**